

Université Euromed de Fès
École d'Ingénierie Digitale et d'Intelligence Artificielle
2^{ème} Année — Intelligence Artificielle

Projet de Fin du Module
Architecture Modernes de Réseaux de Neurones

Rapport de Projet
Audio Understanding
Compréhension Audio par Intelligence Artificielle

Qwen2-Audio-7B-Instruct

Étudiantes : Tassnim & Zineb
Encadrant : Pr. Smail Tigani
Année : 2025 / 2026

Table des matières

1	Introduction	2
1.1	Contexte et motivation	2
1.2	Objectifs du projet	2
2	Méthodologie	3
2.1	Présentation du modèle Qwen2-Audio-7B-Instruct	3
2.1.1	Architecture interne	3
2.1.2	Capacités du modèle	3
2.2	Architecture du système	3
2.3	Déploiement sur Google Colab avec quantification 4 bits	4
2.3.1	Pourquoi la quantification est indispensable	4
2.3.2	Procédure de déploiement	6
2.4	API opérationnelle (FastAPI)	6
2.4.1	Endpoints implémentés	6
2.4.2	Pipeline de traitement	6
2.4.3	Exemple de requête et réponse	7
2.5	Interface web	7
3	Résultats	9
3.1	Fonctionnalités implémentées	9
3.2	Exemples de sorties	9
3.3	Difficultés rencontrées	10
4	Conclusion	11
4.1	Bilan du projet	11
4.2	Pistes d'amélioration	11
	Références bibliographiques	13

1. Introduction

1.1 Contexte et motivation

L'intelligence artificielle connaît depuis quelques années une révolution majeure avec l'émergence des modèles de langage de grande taille (*Large Language Models*, LLMs) et, plus récemment, des modèles multimodaux capables de traiter simultanément plusieurs types de données. Parmi ces avancées, la compréhension audio par IA représente une rupture technologique significative : il ne s'agit plus seulement de transcrire la parole, mais de *comprendre* le contenu sonore dans toute sa richesse sémantique, émotionnelle et contextuelle.

Ce projet s'inscrit dans le cadre du module *Architecture Modernes de Réseaux de Neurones* dispensé à l'École d'Ingénierie Digitale et d'Intelligence Artificielle de l'Université Euromed de Fès. Il vise à concevoir et déployer une plateforme opérationnelle de compréhension audio, en s'appuyant sur un modèle open source de pointe : **Qwen2-Audio-7B-Instruct**, développé par Alibaba Cloud et disponible sur la plateforme Hugging Face.

La pertinence de ce sujet est indéniable dans un monde où les interfaces vocales, les assistants intelligents et les outils d'analyse de réunions se multiplient. Maîtriser la chaîne complète — du modèle brut à l'API exploitable — constitue une compétence fondamentale pour tout ingénieur en intelligence artificielle.

1.2 Objectifs du projet

Le projet poursuit trois objectifs principaux, alignés sur les critères d'évaluation du module :

1. **Monter une API opérationnelle** exposant des points de terminaison POST capables de recevoir des fichiers audio, de les traiter via le modèle Qwen2-Audio-7B-Instruct et de retourner une réponse textuelle structurée.
2. **Développer une mise en application concrète** matérialisée par une interface web interactive permettant la transcription, la compréhension (Q&A) et l'analyse approfondie de contenus audio.
3. **Acquérir une expertise pratique** sur les modèles multimodaux audio-langage en comprenant leur architecture, leur intégration et leurs limites.

2. Méthodologie

2.1 Présentation du modèle Qwen2-Audio-7B-Instruct

Qwen2-Audio-7B-Instruct est un modèle multimodal de compréhension audio développé par le groupe Qwen d'Alibaba Cloud [1]. Il est publié en open source sous licence Apache 2.0 sur Hugging Face et constitue la version affinée par instruction (*instruction-tuned*) du modèle de base Qwen2-Audio-7B.

2.1.1 Architecture interne

Le modèle repose sur une architecture hybride associant deux composants distincts :

- **Un encodeur audio** de type Whisper Large V2 [2], responsable de l'extraction des représentations acoustiques à partir des formes d'onde brutes. Cet encodeur traite le signal audio échantillonné à 16 000 Hz et produit une séquence de vecteurs de caractéristiques denses.
- **Un décodeur de langage Qwen2-7B** [3], un modèle de langage autorégressif à 7 milliards de paramètres, qui reçoit en entrée la concaténation des représentations audio et des tokens textuels, et génère la réponse token par token.

2.1.2 Capacités du modèle

Qwen2-Audio-7B-Instruct excelle dans plusieurs tâches audio-langage : la reconnaissance automatique de la parole (ASR) en plusieurs langues, la réponse à des questions libres portant sur le contenu sonore, l'analyse émotionnelle et tonale, ainsi que la description d'événements sonores non-verbaux (musique, bruits ambiants, etc.).

2.2 Architecture du système

Le système développé s'articule autour de deux couches principales : un backend Python assurant l'inférence du modèle et l'exposition des endpoints, et un frontend HTML/CSS/JS offrant l'interface utilisateur.

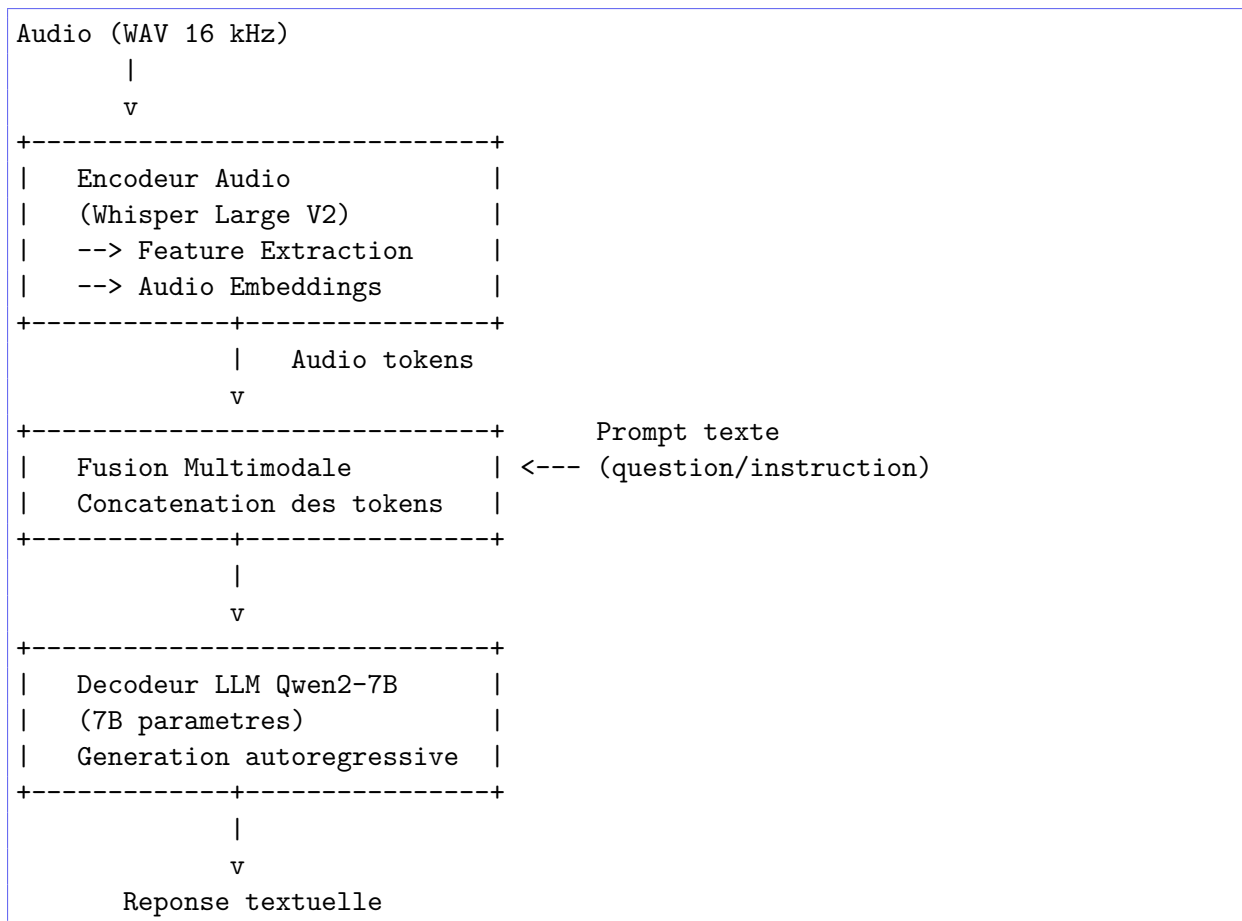


Figure 2.1 – Architecture du modèle Qwen2-Audio-7B-Instruct

2.3 Déploiement sur Google Colab avec quantification 4 bits

Faute de GPU local suffisant, le système est déployé sur **Google Colab** en exploitant un GPU **NVIDIA T4** (15 Go de VRAM), le GPU gratuit le plus courant sur la plateforme.

2.3.1 Pourquoi la quantification est indispensable

Le modèle Qwen2-Audio-7B-Instruct comporte 7 milliards de paramètres. En précision `float16` (2 octets par paramètre), le chargement seul occupe environ $7 \times 10^9 \times 2 = 14$ Go, auxquels s'ajoutent les activations intermédiaires et les buffers du système CUDA, portant le besoin réel à **≈16 Go de VRAM**. Le T4 ne dispose que de 15 Go : le chargement en `float16` est donc impossible sans débordement mémoire.

La **quantification en 4 bits NF4** (*NormalFloat-4*), fournie par la bibliothèque `bitsandbytes`, résout ce problème en ne stockant chaque paramètre que sur un demi-octet, ramenant l'empreinte mémoire à **≈4–5 Go** :

NF4 est un format spécialement conçu pour les distributions gaussiennes typiques des poids des réseaux de neurones. Combiné à la *double quantification* (qui quantifie elle-même

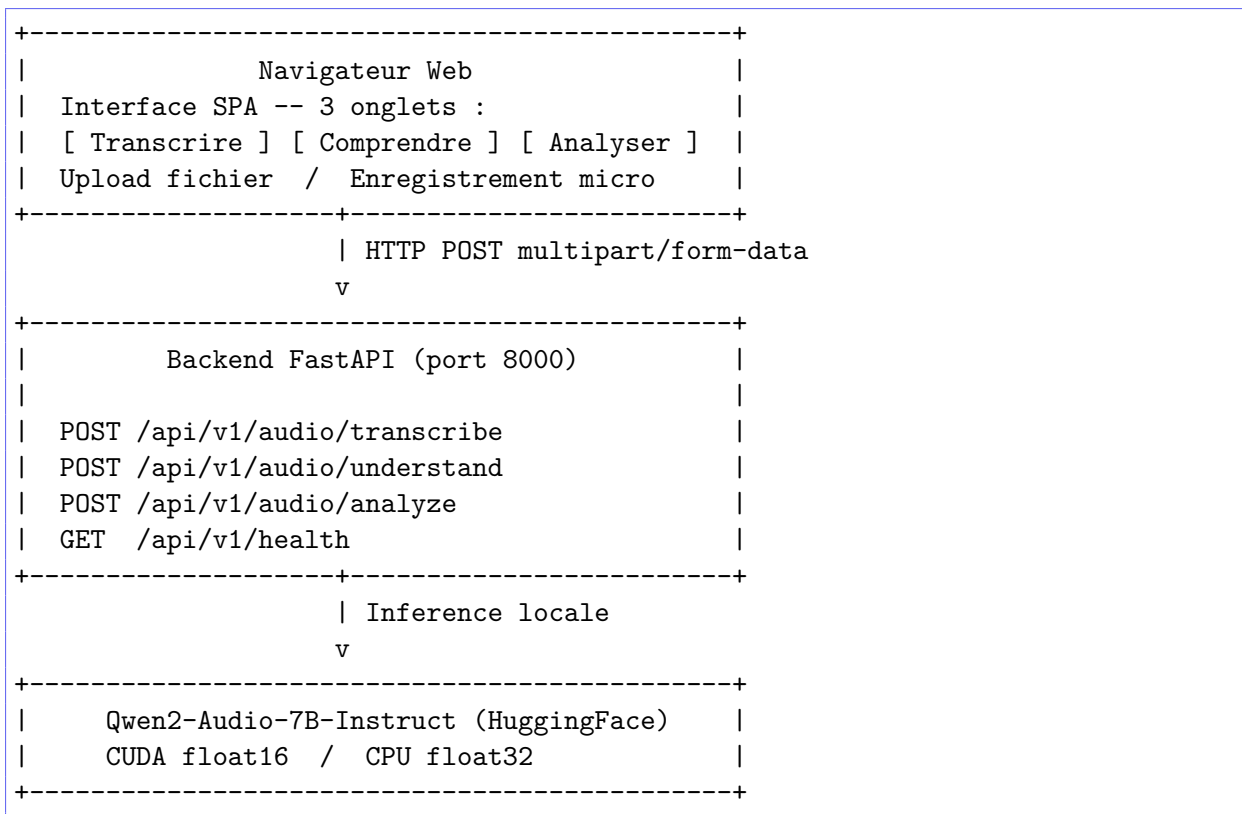


Figure 2.2 – Architecture globale du système

Précision	Octets/paramètre	VRAM requise	GPU Colab compatible
float16 (FP16)	2,0	≈16 Go	A100 (40 Go)
8 bits (INT8)	1,0	≈8 Go	A100, V100
4 bits NF4	0,5	≈5 Go	T4, tout GPU ≥ 6 Go

Table 2.1 – Empreinte mémoire du modèle selon la précision

les constantes de quantification) et à un *compute dtype* en `float16` (les multiplications matricielles restent en demi-précision), la dégradation de qualité reste minimale pour les tâches de compréhension audio.

Dans le code (`backend/app/model.py`), la quantification est activée via la variable d'environnement `LOAD_IN_4BIT=true` :

```

1 from transformers import BitsAndBytesConfig
2
3 quant_config = BitsAndBytesConfig(
4     load_in_4bit=True,
5     bnb_4bit_quant_type="nf4",           # NormalFloat-4
6     bnb_4bit_compute_dtype=torch.float16, # calculs en fp16
7     bnb_4bit_use_double_quant=True,      # double quantification
8 )

```

```
9 model = Qwen2AudioForConditionalGeneration.from_pretrained(  
10     MODEL_NAME ,  
11     quantization_config=quant_config ,  
12     device_map="auto" ,  
13 )
```

Listing 2.1 – Configuration BitsAndBytesConfig pour la quantification NF4

2.3.2 Procédure de déploiement

La procédure est automatisée par le notebook `Qwen2_Audio_Colab.ipynb` :

1. Sélection du GPU T4 dans Colab (*Runtime* → *Change runtime type*).
2. Clonage du dépôt GitHub du projet dans l'environnement Colab.
3. Installation des dépendances (`transformers`, `accelerate`, `bitsandbytes`, `librosa`, `ffmpeg`, `FastAPI`) sans réinstaller PyTorch, déjà fourni avec CUDA par Colab.
4. Activation de la variable `LOAD_IN_4BIT=true` avant le lancement.
5. Lancement du serveur `uvicorn` sur le port 8000.
6. Ouverture d'un tunnel public via `cloudflared`, qui expose l'application sur une URL temporaire `*.trycloudflare.com`.

Comme le serveur FastAPI sert à la fois l'interface web et l'API sur la même origine, l'ouverture de cette unique URL publique suffit à utiliser la plateforme complète depuis n'importe quel navigateur. Le premier démarrage télécharge environ 4–5 Go de poids quantifiés (contre 16 Go en float16) ; les exécutions suivantes utilisent le cache Hugging Face local.

2.4 API opérationnelle (FastAPI)

Le backend est développé avec le framework **FastAPI** [4] (Python), reconnu pour ses performances élevées, sa validation automatique des données via Pydantic, et sa génération automatique de documentation interactive (Swagger UI).

2.4.1 Endpoints implémentés

2.4.2 Pipeline de traitement

Chaque requête suit le pipeline suivant :

1. Réception et validation du fichier audio (formats : WAV, MP3, OGG, FLAC, WebM, M4A ; taille maximale : 50 Mo).
2. Décodage et rééchantillonnage à 16 000 Hz via `soundfile` et `librosa` [5].
3. Construction de la conversation au format *chat template* Qwen2-Audio.

Méthode	Endpoint	Entrée	Sortie
POST	/api/v1/audio/transcribe	Fichier audio	Transcription complète
POST	/api/v1/audio/understand	Fichier audio + question	Réponse à la question
POST	/api/v1/audio/analyze	Fichier audio	Transcription + résumé + sentiment
GET	/api/v1/health	—	État du modèle et du serveur

Table 2.2 – Endpoints de l'API

- Appel au modèle via `model.generate()`.
- Sérialisation de la réponse en JSON.

2.4.3 Exemple de requête et réponse

```

1 curl -X POST http://localhost:8000/api/v1/audio/understand \
2   -F "audio=@reunion.wav" \
3   -F "question=Quels sont les points principaux abordes ?"

```

Listing 2.2 – Requête cURL vers /understand

```

1 {
2   "question": "Quels sont les points principaux abordes ?",
3   "answer": "La reunion porte principalement sur le calendrier du
4             projet, la repartition des taches et le budget
5             alloue
6             pour le trimestre.",
7   "filename": "reunion.wav"
8 }

```

Listing 2.3 – Réponse JSON de l'endpoint /understand

2.5 Interface web

L'interface utilisateur est une application monopage (*Single Page Application*) développée en HTML5, CSS3 et JavaScript vanilla, servie directement par le backend FastAPI. Elle s'organise en trois onglets :

- **Transcrire** — l'utilisateur soumet un fichier audio ou enregistre sa voix directement via le microphone ; le système retourne la transcription intégrale.
- **Comprendre** — l'utilisateur joint un fichier audio et formule une question en langage naturel ; le modèle répond en s'appuyant sur le contenu de l'audio.

- **Analyser** — le système produit automatiquement un rapport complet : transcription, résumé des points clés et analyse du ton/sentiment exprimé.

L'enregistrement audio est réalisé via l'API Web **MediaRecorder** du navigateur, permettant une interaction directe sans logiciel tiers. Le fichier produit est transmis en **multipart/form-data** vers le backend.

3. Résultats

3.1 Fonctionnalités implémentées

L'ensemble des fonctionnalités prévues a été implémenté avec succès. Le tableau ci-dessous récapitule les livrables du projet.

Livrable	Statut	Détails
API POST /transcribe	Implémenté	Multilingue, WAV/MP3/OGG/FLAC/WebM/M4A
API POST /understand	Implémenté	Question libre en langage naturel
API POST /analyze	Implémenté	Transcription + résumé + sentiment
Interface web	Implémenté	3 onglets, upload + enregistrement micro
Documentation Swagger	Auto-générée	Accessible à <code>/api/docs</code>
Validation des entrées	Implémenté	Format, taille, contenu vide
Mode développement	Implémenté	Variable <code>USE MOCK</code> sans GPU

Table 3.1 – Récapitulatif des livrables

3.2 Exemples de sorties

Les tests ont été réalisés avec des fichiers audio variés (parole en français et en anglais, enregistrements de réunions, extraits musicaux).

Transcription — fichier audio de 30 secondes en français

Sortie : « Bonjour à tous. Aujourd'hui nous allons discuter des résultats du dernier trimestre. Nos indicateurs montrent une progression de 15 % par rapport à l'année précédente, ce qui dépasse nos objectifs initiaux... »

Compréhension (Q&A) — Question : « Quelle est l'émotion du locuteur ? »

Réponse : « Le locuteur exprime une satisfaction marquée et un enthousiasme modéré. Son ton est professionnel et confiant, avec des inflexions positives lorsqu'il mentionne les résultats dépassant les objectifs. »

Analyse complète — résumé et sentiment

Résumé : « L'audio présente un discours de bilan trimestriel. Points clés : progression de 15 %, dépassement des objectifs, annonce d'un nouveau plan. »

Sentiment : « Ton positif et professionnel. Sentiment dominant : satisfaction et optimisme. »

3.3 Difficultés rencontrées

Le développement du projet a présenté plusieurs défis techniques :

- **Gestion des formats audio hétérogènes.** Les navigateurs web produisent des enregistrements au format WebM/Opus, non nativement supporté par `soundfile`. La solution adoptée consiste à utiliser `librosa` comme *fallback*, qui s'appuie sur `ffmpeg` pour décoder les formats exotiques.
- **Contrainte mémoire GPU.** Le modèle Qwen2-Audio-7B-Instruct requiert environ 16 Go de VRAM en précision `float16`, ce qui dépasse la capacité du GPU T4 disponible sur Google Colab (15 Go). La solution retenue est la **quantification NF4 4 bits** via `bitsandbytes`, qui ramène l'empreinte mémoire à environ 4-5 Go tout en conservant une qualité de transcription et de compréhension satisfaisante. La variable d'environnement `LOAD_IN_4BIT=true` active cette configuration au démarrage.
- **Audio non transmis au modèle.** Le défi technique le plus subtil a été un bug où le modèle répondait comme un simple modèle de langage textuel (« je ne peux pas accéder à l'audio »), sans aucune erreur apparente. La cause : l'argument du processeur transportant l'audio a été **renommé** d'une version de `transformers` à l'autre (`audios` → `audio`). Lorsqu'on passe le mauvais nom, l'argument est silencieusement ignoré (absorbé par `**kwargs`) et l'audio disparaît des entrées du modèle. **Solution** : essayer successivement les deux noms et ne conserver que l'appel produisant effectivement des caractéristiques audio (`input_features`), échouer explicitement sinon, et aligner le type des tenseurs audio (`float16`) sur celui du modèle.
- **Gestion du chat template multimodal.** La méthode `apply_chat_template` du processeur Qwen2-Audio nécessite une structure de conversation précise mélangeant tokens audio et tokens textuels, dont la compréhension a demandé une lecture approfondie du code source du modèle.
- **Temps d'initialisation.** Le chargement du modèle depuis Hugging Face représente environ 16 Go de téléchargement à la première exécution. Le modèle est ensuite mis en cache localement pour les exécutions suivantes.

4. Conclusion

4.1 Bilan du projet

Ce projet nous a permis de maîtriser la chaîne de valeur complète d'un système d'IA audio en production. Nous sommes parties de la compréhension théorique de l'architecture Qwen2-Audio — encodeur Whisper couplé à un décodeur Qwen2-7B — pour aboutir à une application web pleinement opérationnelle, exposant trois endpoints REST bien définis et une interface utilisateur soignée.

Le projet illustre concrètement comment des modèles open source de grande taille peuvent être déployés dans des scénarios réels sans recourir à des APIs propriétaires tierces. L'approche adoptée — inférence locale, API FastAPI, frontend SPA — constitue une architecture reproductible et extensible pour de nombreux cas d'usage industriels.

Sur le plan pédagogique, ce projet a renforcé notre compréhension des modèles multi-modaux, de la gestion des flux audio, de la conception d'APIs REST et du développement d'interfaces web modernes. Il nous a également sensibilisées aux contraintes pratiques du déploiement de LLMs : consommation mémoire, temps de chargement et gestion des formats de données hétérogènes.

4.2 Pistes d'amélioration

Plusieurs améliorations pourraient enrichir ce projet dans une version future :

- **Streaming des réponses.** Implémenter les *Server-Sent Events* (SSE) pour afficher la réponse du modèle token par token, améliorant ainsi la réactivité perçue par l'utilisateur.
- **Conversation multi-tours.** Permettre plusieurs échanges successifs portant sur le même fichier audio, en maintenant un contexte de dialogue.
- **Détection de langue automatique.** Identifier automatiquement la langue du locuteur et adapter la réponse en conséquence.
- **Support des audios longs.** Implémenter un mécanisme de découpage (*chunking*) pour traiter des enregistrements de plusieurs heures (réunions, conférences, podcasts).

- **Quantification avancée.** Au-delà de la quantification 4/8 bits (`bitsandbytes`) déjà prise en charge, explorer des méthodes hors ligne telles que GPTQ ou AWQ pour réduire davantage l’empreinte mémoire et accélérer l’inférence sur des GPUs de faible capacité.
- **Évaluation formelle (WER).** Mesurer le *Word Error Rate* sur un corpus francophone de référence pour quantifier objectivement la précision du modèle.

Références bibliographiques

- [1] Yunfei CHU et al. “Qwen2-Audio Technical Report”. In : *arXiv preprint arXiv :2407.10759* (2024). URL : <https://arxiv.org/abs/2407.10759>.
- [2] Alec RADFORD et al. “Robust Speech Recognition via Large-Scale Weak Supervision”. In : *Proceedings of the 40th International Conference on Machine Learning (ICML)*. 2023. URL : <https://arxiv.org/abs/2212.04356>.
- [3] An YANG et al. “Qwen2 Technical Report”. In : *arXiv preprint arXiv :2407.10671* (2024). URL : <https://arxiv.org/abs/2407.10671>.
- [4] Sebastián RAMÍREZ. *FastAPI : Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com>. Consulté en 2025. 2024.
- [5] Brian MCFEE et al. “librosa : Audio and Music Signal Analysis in Python”. In : *Proceedings of the 14th Python in Science Conference* (2015). URL : <https://librosa.org>.